Техническое задание (ТЗ): Проект «Apple Axiom» (ИИ-ассистент / Виртуальный питомец)

1. Концепция проекта

Apple Technik — это интерактивный ИИ-ассистент (виртуальный робот-питомец), встроенный в операционную систему устройств Apple (iOS / macOS). Он объединяет функции системной утилиты, предиктивной диагностики оборудования и геймификации, заботясь о состоянии «железа» и эмоционально взаимодействуя с пользователем.

---

2. Основные компоненты и доработки идеи

2.1. Геймификация и эмпатия (Характер и «живой» интерфейс)

- **Состояние устройства = Настроение персонажа:**
    - При нормальной работе устройства (чистый диск, высокая емкость батареи, отсутствие перегрева) робот бодр, счастлив, меняет анимации.
    - При проблемах (забитые кулеры, износ аккумулятора, троттлинг, нехватка памяти) робот «кашляет», грустит, меняет аватарку или статус в Dynamic Island / меню-баре.
- **Визуальный стиль:** Дружелюбный кастомный питомец в духе современных тамагочи или роботов из Wall-E / Astro Bot (отход от строгого технократичного интерфейса).
- **Геймификация ухода:** Очистка кэша, удаление мусора или оптимизация системы преподносятся как «лечение» питомца, за которое он «радуется» и открывает кастомные скины/анимации.

2.2. Умная диагностика и предиктивная аналитика

- **Цифровой стетоскоп (Аудиодиагностика):** Использование микрофона устройства для анализа шумов кулера, свиста или треска накопителей с выдачей понятного описания проблемы.
- **Предиктивный анализ батареи:** Прогнозирование деградации аккумулятора в зависимости от сценариев использования и предложения по включению умной защиты.
- **Открытая экосистема (GitHub / Open Source модуль):** Модульная архитектура, позволяющая разработчикам создавать собственные плагины-диагносты для специфических задач.

2.3. Автоматизация и исправление (Self-Healing)

- **Интерактивные скрипты оптимизации:** Кнопки быстрого действия для безопасной очистки кэша (например, кэша Xcode, неиспользуемых симуляторов) с освобождением гигабайтов памяти.
- **Автоматическая настройка параметров:** Предложения и автоматическое применение временного снижения пиковой частоты процессора или изменения режима энергосбережения при перегреве.

---

3. Архитектура и технологии

- **Кроссплатформенное ядро:** Планируется использование Kotlin Multiplatform (KMP) для написания бизнес-логики анализа данных, симуляции состояния устройства и бэкенд-архитектуры.
- **Интерфейс (UI/UX):** Нативные компоненты под iOS (SwiftUI, Dynamic Island, Widgets) и macOS (Menu Bar, Floating Window).

---

4. Этапы разработки (Roadmap)

1. **MVP на GitHub:** Создание репозитория, публикация детального Readme и концепт-артов персонажа.
2. **Разработка ядра (KMP):** Реализация логики мониторинга и симуляции состояния «здоровья» устройства.
3. **Интерфейс и интеграция:** Создание анимаций питомца и интеграция сценариев диагностики (на примере батареи и перегрева).